

EXPERIMENT 20

IMPLEMENTATION OF DIGITAL SIGNATURE

AIM:

To implement a Digital Signature using the RSA algorithm in C and demonstrate message signing and signature verification.

ALGORITHM:

Key Generation

1. Select two prime numbers p and q .
2. Calculate:

$$n = p \times q$$

3. Calculate Euler's totient:

$$\phi(n) = (p - 1) \times (q - 1)$$

4. Select a public exponent e such that:

$$\gcd(e, \phi(n)) = 1$$

5. Calculate the private key d such that:

$$(d \times e) \bmod \phi(n) = 1$$

6. The public key is (e, n) and the private key is (d, n) .

Signature Generation

1. Enter the message as an integer smaller than n .
2. Generate the digital signature using the private key:

$$\text{Signature} = \text{Message}^d \bmod n$$

Signature Verification

1. Use the public key to verify the signature:

$$\text{Verified Message} = \text{Signature}^e \bmod n$$

2. Compare the verified message with the original message.
3. If both are equal, the digital signature is valid.

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
/* Greatest Common Divisor */
```

```
int gcd(int a, int b)
```

```
{
```

```
    while (b != 0)
```

```
    {
```

```
        int temp = b;
```

```
        b = a % b;
```

```
        a = temp;
```

```
    }
```

```
    return a;
```

```
}
```

```
/* Modular exponentiation */
```

```
long long powerMod(long long base,
```

```
                    long long exponent,
```

```
                    long long modulus)
```

```
{
```

```
    long long result = 1;
```

```
    base = base % modulus;
```

```
    while (exponent > 0)
```

```
    {
```

```
        if (exponent % 2 == 1)
```

```

    {
        result =
            (result * base) % modulus;
    }

    base =
        (base * base) % modulus;

    exponent = exponent / 2;
}

return result;
}

/* Find modular inverse */
int modularInverse(int e, int phi)
{
    int d;

    for (d = 1; d < phi; d++)
    {
        if ((e * d) % phi == 1)
            return d;
    }

    return -1;
}

int main()

```

```

{
    int p, q;
    int n, phi;
    int e, d;
    long long message;
    long long signature;
    long long verifiedMessage;

    printf("DIGITAL SIGNATURE USING RSA\n");
    printf("-----\n");

    printf("Enter prime number p: ");
    scanf("%d", &p);

    printf("Enter prime number q: ");
    scanf("%d", &q);

    /* Calculate n */
    n = p * q;

    /* Calculate Euler's totient */
    phi = (p - 1) * (q - 1);

    printf("Enter value of e: ");
    scanf("%d", &e);

    /* Check e */
    if (gcd(e, phi) != 1)
    {

```

```

    printf("\nInvalid value of e.\n");
    printf("e must be relatively prime to phi.\n");
    return 0;
}

/* Calculate private key */
d = modularInverse(e, phi);

if (d == -1)
{
    printf("\nPrivate key could not be calculated.\n");
    return 0;
}

printf("\nRSA Parameters\n");
printf("n = %d\n", n);
printf("phi = %d\n", phi);

printf("Public Key = (%d, %d)\n", e, n);
printf("Private Key = (%d, %d)\n", d, n);

printf("\nEnter message (integer less than %d): ", n);
scanf("%lld", &message);

if (message < 0 || message >= n)
{
    printf("Invalid message.\n");
    return 0;
}

```

```
/* Generate digital signature */
signature =
    powerMod(message, d, n);

/* Verify digital signature */
verifiedMessage =
    powerMod(signature, e, n);

printf("\nDIGITAL SIGNATURE\n");
printf("-----\n");

printf("Original Message  : %lld\n",
    message);

printf("Digital Signature : %lld\n",
    signature);

printf("Verified Message  : %lld\n",
    verifiedMessage);

if (message == verifiedMessage)
{
    printf("\nSignature Verification: VALID\n");
}
else
{
    printf("\nSignature Verification: INVALID\n");
}
```

```

return 0;

}

```

OUTPUT:

The screenshot shows a C program running in a terminal. The program prompts the user to enter prime numbers p and q, and a value e. It then calculates RSA parameters: n = 323, phi = 288, Public Key = (5, 323), and Private Key = (173, 323). The user enters a message of 7. The program generates a digital signature of 125 and prints the original message, digital signature, and verified message (7). The signature verification is successful.

```

main.c Run
58 {
128 printf("Original Message : %ld\n",
131 printf("Digital Signature : %ld\n",
132 signature);
133
134 printf("Verified Message : %ld\n",
135 verifiedMessage);
136
137 if (message == verifiedMessage)
138 {
139 printf("\nSignature Verification: VALID\n");
140 }
141 else
142 {
143 printf("\nSignature Verification: INVALID\n");
144 }
145
146 return 0;
147 }

```

```

Output
DIGITAL SIGNATURE USING RSA
-----
Enter prime number p: 17
Enter prime number q: 19
Enter value of e: 5

RSA Parameters
n = 323
phi = 288
Public Key = (5, 323)
Private Key = (173, 323)

Enter message (integer less than 323): 7

DIGITAL SIGNATURE
-----
Original Message : 7
Digital Signature : 125
Verified Message : 7

```

The screenshot shows the same C program running in a terminal. It displays the RSA parameters and the user's input for the message (7). The program then prints the digital signature (125) and the verified message (7). The signature verification is successful, and the program prints "Signature Verification: VALID".

```

main.c Run
58 {
128 printf("Original Message : %ld\n",
131 printf("Digital Signature : %ld\n",
132 signature);
133
134 printf("Verified Message : %ld\n",
135 verifiedMessage);
136
137 if (message == verifiedMessage)
138 {
139 printf("\nSignature Verification: VALID\n");
140 }
141 else
142 {
143 printf("\nSignature Verification: INVALID\n");
144 }
145
146 return 0;
147 }

```

```

Output
RSA Parameters
n = 323
phi = 288
Public Key = (5, 323)
Private Key = (173, 323)

Enter message (integer less than 323): 7

DIGITAL SIGNATURE
-----
Original Message : 7
Digital Signature : 125
Verified Message : 7

Signature Verification: VALID

==> Code Execution Successful ==>

```

RESULT:

Thus, the Digital Signature algorithm using RSA was successfully implemented in C. The message was signed using the private key, and the signature was successfully verified using the corresponding public key.